# StatQuest: An Interactive Python Game for Practicing Core Statistics Concepts

Kai Steinke
*Statistics for Computer Science, Advanced*
*Lucerne University of Applied Sciences and Arts*
Rotkreuz, Switzerland

Mauro Zappa
*Statistics for Computer Science, Advanced*
*Lucerne University of Applied Sciences and Arts*
Rotkreuz, Switzerland

***Abstract*—StatQuest is an interactive Python application that turns core statistics practice into a short browser-based game. The project was developed for the course Statistics for Computer Science, Advanced, whose goals include applying statistical methods with Python, interpreting statistical results, and communicating data-based conclusions to an audience. The application focuses on three concepts that are important for later data analysis: Pearson correlation, distribution shape, and outlier detection. Players choose a difficulty level, complete five randomized rounds in one of three modes, receive immediate feedback, and can save their result to a local leaderboard. The project combines statistical simulation, visual analytics, and user interaction in Streamlit, using NumPy, SciPy, and Plotly to generate data and display it clearly. By requiring students to estimate, classify, and justify statistical patterns visually, StatQuest supports active learning rather than passive memorization.**

***Index Terms*—statistics education, Python, Streamlit, correlation, distributions, outliers, data visualization**

## I. Introduction

Statistics is easiest to understand when learners can connect formulas to visible patterns in data. In a programming-focused statistics course, this creates an opportunity to combine conceptual learning with practical Python development. StatQuest was designed around this idea: it is not a calculator that hides statistical reasoning, but a game that repeatedly asks the player to inspect a plot, make a decision, and compare that decision with a computed statistical result.

The course syllabus emphasizes the relevance of statistics in computing careers, Python-based data analysis, correct interpretation of results, and clear communication for the audience. StatQuest addresses these goals through a lightweight application that can be used during a classroom demonstration or independently for practice. The project also fits the option of creating an application or game in Python, while still remaining closely connected to statistical content.

## II. Project Design

The application is structured as a five-round mini-game. At the start, the player selects a difficulty level: easy, medium, or hard. The difficulty changes the number of generated observations, using 30, 60, or 100 data points. Larger samples make some visual patterns more stable, but the hard setting also introduces more subtle statistical differences, especially in the correlation task.

After selecting the difficulty, the player chooses one of three game modes. In *Guess the Correlation*, the player estimates Pearson's correlation coefficient from a scatter plot using a slider from -1 to 1. In *Name That Distribution*, the player identifies a histogram as normal, right-skewed, left-skewed, bimodal, or uniform. In *Spot the Outlier*, the player clicks the point that appears most unusual in a strip plot. Each mode gives immediate feedback after submission and then advances to the next randomized round.

The game design is intentionally short. Five rounds are enough to show repeated practice and scoring, but short enough for a live class presentation. The results screen summarizes the final score, the number of successful rounds, the average score per round, and a round-by-round chart. A local leaderboard lets players compare results by initials, score, mode, and difficulty.

## III. Statistical Concepts

The first concept is Pearson's correlation coefficient, commonly denoted by $r$. StatQuest generates paired observations from a bivariate normal distribution with a selected target correlation. The actual sample correlation is computed with SciPy and shown after the player submits an estimate. This distinction between the target parameter and the realized sample statistic is important: even when the data are generated with a known relationship, finite samples contain random variation.

The second concept is distribution shape. The application samples from several distributions that produce recognizable visual forms: normal, right-skewed, left-skewed, bimodal, and uniform. The player must identify the shape from a histogram, while the feedback explains features such as symmetry, skewness, long tails, and multiple peaks. The task trains pattern recognition that is useful before applying more advanced methods such as hypothesis tests, regression, or classification.

The third concept is outlier detection. The application generates normally distributed data and injects one extreme observation. It then computes standardized z-scores for all points. If the player clicks the exact injected outlier, full credit is awarded. If the selected point is not the injected point but still has a large absolute z-score, partial credit is possible. This scoring reflects the practical idea that outlier detection is often

a matter of evidence and thresholds rather than only a binary label.

## IV. Implementation

StatQuest is implemented as a Streamlit application. Streamlit was chosen because it allows Python code to produce a usable browser interface without requiring a separate frontend framework. The application is organized into separate view modules for the home screen, mode selection, each game mode, and the results page. Shared statistical logic is separated into generator, scorer, and feedback modules.

NumPy is used for random sampling and numerical arrays. SciPy is used to compute Pearson's correlation coefficient and skewness. Plotly creates the interactive scatter plots, histograms, strip plots, and result charts. Streamlit session state stores the current page, score, selected difficulty, round number, generated data, submitted answers, and leaderboard status. This state management makes the game feel continuous across user interactions even though Streamlit reruns the script after each input.

The scoring system is mode-specific. Correlation scoring rewards closeness to the true sample correlation, with descriptive labels such as Excellent, Good, Close, and Off. Distribution scoring gives credit for the correct class. Outlier scoring gives full or partial credit depending on the selected point's relationship to the true outlier and its z-score. This makes the evaluation transparent and aligned with the statistical task in each mode.

## V. Learning Value

StatQuest supports learning in three ways. First, it gives immediate feedback. After every answer, the player sees the correct value or category, the points earned, and a short explanation of the relevant statistical idea. Second, it uses random data generation, so repeated attempts do not simply train memorization of fixed examples. Third, it connects numerical statistics to visual reasoning, which is essential for real data analysis.

The project also demonstrates Python programming skills that are relevant for data science. It includes modular code organization, reusable functions for data generation and scoring, interactive visualization, user input handling, persistent JSON-based leaderboard storage, and a styled user interface. These implementation choices make the project more than a static notebook: it is a complete small application with a clear audience and a repeatable user workflow.

## VI. Conclusion

StatQuest combines statistics practice and Python programming in a form that is accessible for classroom presentation. The game covers correlation, distribution shape, and outlier detection through randomized visual tasks, immediate feedback, and score-based motivation. It supports the course goals by requiring statistical interpretation, Python-based implementation, and communication of results in a user-friendly format. Future extensions could add modes for confidence intervals, hypothesis testing, regression, clustering, or classification, which would align with later topics from the course syllabus.